

MAT21-6

Graph Theory

Graphs, matrices, traversals, weighted algorithms, and PageRank

Graph-theoretic thinking with explicit linear-algebra formalization.

Albert School — 18 hours

1 Graph foundations

Definition 1 A **simple graph** is a pair $G = (V, E)$ where V is a finite set of **vertices** and E is a set of unordered pairs $\{u, v\}$ of distinct vertices, called **edges**. A **directed graph** (digraph) replaces unordered pairs by ordered pairs (u, v) , called **arcs**. A **weighted graph** assigns to each edge a real number $w(e)$, its **weight**.

Definition 2 In an undirected graph, the **degree** $\deg(v)$ of a vertex v is the number of edges incident to v . In a digraph, the **in-degree** $\deg^-(v)$ and **out-degree** $\deg^+(v)$ count the arcs entering and leaving v .

Definition 3 A **walk** of length k is a sequence $v_0, v_1, \dots, v_k$ of vertices with $\{v_{i-1}, v_i\} \in E$ for each i . A **path** is a walk with no repeated vertex. A **cycle** is a walk of length ≥ 1 with $v_0 = v_k$ and no other repeated vertex. A graph is **connected** if there is a path between every pair of vertices; its **connected components** are the maximal connected subgraphs.

Definition 4 A **tree** is a connected graph with no cycles. A **forest** is a graph whose components are trees.

2 Representations and linear-algebra review

Definition 5 Let $V = \{v_1, \dots, v_n\}$. The **adjacency matrix** $A \in \mathbb{R}^{n \times n}$ has $A_{ij} = 1$ if $\{v_i, v_j\} \in E$ (or $(v_i, v_j) \in E$ for a digraph) and $A_{ij} = 0$ otherwise. For an undirected graph A is symmetric. For a weighted graph one stores $A_{ij} = w(\{v_i, v_j\})$ instead of 1.

For an undirected simple graph, the i -th row sum of A equals $\deg(v_i)$, so the degree sequence and (for weighted graphs) the edge weights are read directly off A .

Definition 6 The **incidence matrix** $B \in \mathbb{R}^{n \times m}$ of an undirected graph with m edges has $B_{ie} = 1$ if vertex v_i is an endpoint of edge e , and 0 otherwise. The **degree matrix** D is the diagonal matrix with $D_{ii} = \deg(v_i)$.

2.1 The handshake lemma

Theorem 7 (Handshake lemma) For any undirected graph $G = (V, E)$,

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Proof. Form the sum by counting, over all vertices v , the edges incident to v . Each edge $\{u, w\}$ has exactly two endpoints, so it is counted exactly twice in the left-hand sum: once at u and once at w . Hence the total equals $2|E|$. $\square$

Corollary 8 The number of vertices of odd degree is even. Consequently a degree sequence whose entries sum to an odd number, or that has an odd number of odd entries, cannot be realized by any graph.

2.2 Counting walks with matrix powers

Theorem 9 Let A be the adjacency matrix of a graph. For every integer $k \geq 0$, the entry $(A^k)_{ij}$ equals the number of walks of length k from v_i to v_j .

Proof. By induction on k . For $k = 1$, $(A^1)_{ij} = A_{ij}$ is the number of walks of length 1 (i.e. edges) from v_i to v_j . Assume the claim for k . Then

$$(A^{k+1})_{ij} = \sum_{l=1}^n (A^k)_{il} A_{lj}.$$

A walk of length $k + 1$ from v_i to v_j consists of a walk of length k from v_i to some vertex v_l followed by an edge $\{v_l, v_j\}$. The term $(A^k)_{il} A_{lj}$ counts exactly those walks passing through v_l at step k ; summing over l counts all of them. This is why the matrix-product formula counts walks. $\square$

2.3 The graph Laplacian

Definition 10 For an undirected graph the **Laplacian** is $L = D - A$, where D is the degree matrix and A the adjacency matrix.

Proposition 11 L is symmetric and positive semi-definite.

Proof. Symmetry: D is diagonal and A is symmetric, so $L = D - A$ is symmetric. For positive semi-definiteness, let $x \in \mathbb{R}^n$. Then

$$x^T L x = \sum_i D_{ii} x_i^2 - \sum_{i,j} A_{ij} x_i x_j = \sum_{\{i,j\} \in E} (x_i - x_j)^2 \geq 0,$$

where the last equality groups the terms edge by edge: each edge $\{i, j\}$ contributes $x_i^2 + x_j^2 - 2x_i x_j = (x_i - x_j)^2$. As a sum of squares this is non-negative, so L is positive semi-definite. $\square$

3 Traversal and structure algorithms

3.1 Breadth-first and depth-first search

Definition 12 **Breadth-first search (BFS)** from a source s visits s , then all vertices at distance 1, then distance 2, and so on, using a FIFO queue. **Depth-first search (DFS)** explores as far as possible along each branch before backtracking, using a stack (or recursion). Each records a **traversal tree** whose edges are those along which a new vertex was first discovered.

To run BFS by hand from s : enqueue s and mark it visited; repeatedly dequeue a vertex, and for each unvisited neighbour mark it visited, record the tree edge, and enqueue it. DFS is the same with a stack (or recursion), exploring one neighbour fully before the next.

A graph is **connected** iff a single BFS or DFS from any vertex reaches all vertices; running the traversal once per unvisited vertex enumerates the connected components.

Proposition 13 (BFS computes unweighted shortest paths) In an unweighted graph, BFS from s labels each vertex v with a distance $d(v)$ equal to the length of a shortest path from s to v .

Proof. BFS processes vertices in non-decreasing order of their assigned label, and a vertex first discovered from a vertex at label d receives label $d + 1$. By induction on d , every vertex with label d is reachable by a walk of length d , so its true distance is $\leq d$. Conversely a vertex at true distance d has a neighbour at true distance $d - 1$, which by induction is labelled $d - 1$ and is processed before any vertex of larger label, so the vertex is labelled d . Hence the label equals the shortest-path length. $\square$

3.2 Bipartite graphs

Definition 14 A graph is **bipartite** if V can be split into two sets X, Y such that every edge joins a vertex of X to a vertex of Y — equivalently, if it admits a proper **2-colouring**.

Theorem 15 A graph is bipartite if and only if it contains no cycle of odd length.

Proof. If G is bipartite with parts X, Y , any cycle alternates between X and Y , so it has even length; hence no odd cycle exists. Conversely, suppose no odd cycle exists; on each component run BFS from a vertex s and colour each vertex by the parity of its BFS distance. An edge between two vertices of equal parity would close a cycle of odd length, a contradiction; so every edge joins the two parities and the colouring is proper. $\square$

This gives the **verification procedure**: BFS-colour by distance parity; the graph is bipartite iff no edge joins two vertices of the same colour.

3.3 Trees

Theorem 16 (Characterizations of trees) For a graph G with n vertices, the following are equivalent:

1. G is connected and acyclic;
2. G is connected and has exactly $n - 1$ edges;
3. between every pair of vertices there is exactly one path.

Proof. $(1 \Rightarrow 3)$ Connectivity gives at least one path between any two vertices; two distinct paths would together contain a cycle, contradicting acyclicity, so the path is unique. $(3 \Rightarrow 1)$ A unique path between every pair implies connectivity; a cycle would give two distinct paths between two of its vertices, so G is acyclic. $(1 \Leftrightarrow 2)$ A connected acyclic graph on n vertices has $n - 1$ edges (by induction, removing a leaf), and a connected graph with $n - 1$ edges has no cycle, since removing any cycle edge keeps it connected and would force fewer than $n - 1$ edges. Thus all three are equivalent. $\square$

To check whether a given graph is a tree, verify any one characterization (e.g. connected via a single BFS/DFS and edge count = $n - 1$). In a tree the unique path between two vertices is the shortest path; it is found by BFS/DFS from one endpoint, reading off the path in the traversal tree.

4 Weighted-graph essentials

4.1 Dijkstra's algorithm

Definition 17 **Dijkstra's algorithm** computes shortest-path distances from a source s in a graph with **non-negative** edge weights. It maintains a tentative distance for every vertex; repeatedly it extracts the unsettled vertex u of minimum tentative distance, settles it, and **relaxes** each edge (u, v) : if $d(u) + w(u, v) < d(v)$, set $d(v) = d(u) + w(u, v)$.

The non-negativity assumption is what justifies settling the minimum-distance vertex permanently: with non-negative weights, no later path through a not-yet-settled vertex can improve on the current minimum, since extending a path can only add non-negative length. A **priority queue** keyed on tentative distance supplies the minimum-distance vertex efficiently at each step, which is why the algorithm relies on one.

4.2 Minimum spanning trees and Kruskal's algorithm

Definition 18 A **spanning tree** of a connected graph is a subgraph that is a tree and contains all vertices. A **minimum spanning tree (MST)** of a weighted graph is a spanning tree of minimum total edge weight.

Definition 19 **Kruskal's algorithm** sorts the edges by non-decreasing weight and considers them in order, adding an edge to the tree whenever it joins two different components (i.e. does not create a cycle), until $n - 1$ edges are chosen.

The cycle test — does this edge join two distinct components? — is what keeps the result acyclic; after $n - 1$ successful additions the chosen edges form a spanning tree, and choosing the lightest safe edge at each step yields one of minimum total weight.

5 Data/AI bridge: random walks and PageRank

Definition 20 A **random walk** on a graph moves from the current vertex to a uniformly chosen neighbour at each step. Its **transition matrix** P has $P_{ij} = 1/\deg(v_i)$ if $\{v_i, v_j\} \in E$ and 0 otherwise, so every row sums to 1 (P is **row-stochastic**). If $\pi^{(t)}$ is the row vector of vertex probabilities at time t , then $\pi^{(t+1)} = \pi^{(t)}P$.

Definition 21 Model the web as a directed graph of pages and links. The **PageRank** vector π is the **stationary distribution** of the associated stochastic matrix P , i.e. the probability row vector satisfying

$$\pi = \pi P, \quad \sum_i \pi_i = 1.$$

Writing the condition as $\pi^T = P^T \pi^T$, the PageRank vector is (up to normalization) the eigenvector of P^T for eigenvalue 1 — the **dominant eigenvector** of the transition operator. It is computed by **power iteration**: start from any probability vector $\pi^{(0)}$ and repeatedly apply $\pi^{(t+1)} = \pi^{(t)} P$; under standard assumptions the iterates converge to π . One or two steps are carried out by hand, or in Python:

```
import numpy as np
# P: row-stochastic transition matrix (n x n)
pi = np.ones(P.shape[0]) / P.shape[0]
for _ in range(2):          # one or two power-iteration steps
    pi = pi @ P
```

Scope covered. Graph foundations (simple/directed/weighted graphs, degree, paths, cycles, connectivity, components, trees); adjacency/incidence/degree matrices, Laplacian, walk counting via A^k ; BFS/DFS, connectedness, bipartite check, unweighted shortest paths; Dijkstra, Kruskal MST, priority queues; random walks, transition matrices, and PageRank as a stochastic-matrix fixed point.